Operating Systems

Restaurant (Project 5)

Alessio Zanon
Lorenzo Meduri

August 29, 2026

Contents

1 Description

The Restaurant itself is responsible for:

- Loading environment variables
- Loading resources and the menu from data files (menu.csv and resources.csv)
- Thread arguments creation, through **CookArg**, **WaiterArg**, **CustomerArg**, e.g. pipes, mutexes/semaphores, random seeds....
- Spawning and joining the threads

1.1 Actors

There are three types of actors, each with its own thread type

1.1.1 Customers

Customers generete their order randomly and place it in one slot of the shared memory with the waiters, then announce their slot's position in the pipe they share with them. They also randomly generate their patience, within the range (patienceFloor, patienceFloor + PATIENCE_LEVEL_RANGE), with patienceFloor being the minimum time needed to satisfy their order. They repeatedly poll their order completion and wait game time. They either leave the restaurant because their patience ran out or because their order was completed, updating the restaurant's score accordingly.

1.1.2 Waiters

Waiters are notified of a customer arrival via a one-way many-to-many pipe, then read the dishes involved in their clients' orders from a shared allocated memory. The Waiter then chooses the best Cook for each dish, and sends the relevant information via a one-way pipe. When not handling customer arrivals, waiters poll their assigned Customer-to-Waiter pipe and attempt to entartain the Customer with lowest patience.

1.1.3 Cooks

Cooks receive dishes to prepare by reading a one-way many-to-one pipe, and upon completion of the dish write to the one-way many-to-one pipe with the waiter as receiver. Each Cook is assigned an atomic integer, shared with all waiters, that is used to keep track of their queue.

Cooks receive dishes to prepare by reading a one-way many-to-one pipe, and upon completion of the dish write to the one-way many-to-one pipe with the waiter as receiver. Each Cook is assigned an atomic integer, shared with all waiters, that is used to keep track of their queue.

2 Design Choices

2.1 Randomization

The seed taken from .env is used as seed for the Splitmix64 library, which is used to generate each thread's seed by Restaurant, passing them through the arg structs. Each thread then uses the xoshiro256plusplus library to generate all their random numbers. Since the latter library by default keeps a state (the seed) used by the function next(), we modified it to take the state as argument to the function, making it possible for each thread to use its own seed. All library used are pseudo-random, meaning that given the same initial seed, their generated numbers will be the same.

2.2 Extra-thread Communication

Comunication among threads is handled in two ways: Shared Memory and one-way Pipes

Shared Memory is used for all data that might need to be read or modified multiple times, like the "run" boolean for waiters and cooks, the order array, the cook's queue time array, and the client status.

When data instead was inteded as single-use, Pipes were chosen instead. Their main benefit was the inherent thread-safety and consistency they provide, as their operations are atomic (within the bounds of some sizeable bit amount) and the removal-on-read guarantees that even on -to-many pipes only one thread will read any one message. This way, the operating system shoulders the burden to guarantee that only one waiter attends any client, and that mutliple cook-to-waiter or waiter-to-cook messages don't collide and are instead resolved orderly by each thread.

2.3 Waiter-Cook Scheduling

In order to maximise parallelism, every Waiter immediately assignes all the orders of their new client, using the following metrics in order of patience, speed of dish, price of dish. Each dish is then assigned to the currently least busy cook (as they're the ones most likely to complete it first). Each cook then looks at their queued dishes, and tries to fullfill them by order received, with some caveats. The cook runs a first iteration where all dishes that cannot be done only with clean resources are ignored. This is to prevent a scenarion in simple FIFO where score is detracted due to dirty-use when the cook could do something else aproximately as productive without such drawbacks. Failing that, the cook examines their queue lenght, comparing it to a constant value decided at compile time to deicde whether it is "overworked". If the cook is under the overworked threshold, it is considered the optimal choice to attempt to clean a reource if the sink is free, or even idle to not incur a score penalty. If the cook is over the threshold, then it will attempt to prepare the oldest dish in the queue where the sum of dirty and clean resources is sufficient. In this case, the cook will still attempt to use as many clean resources as possible, and the least used ones of among the dirty.

A simple algorithm is used to decide what resource to clean by a cook. The reource with the highest priority is the one with the least clean units left (implying it is either a scarce resource, or a plentiful resource requently used) and least amount of dirty units left as a tie breaker. Of course, once a reource is chosen, the one with the greatest penalty is cleaned.

Once a dish is prepared, the cook sends it back to the waiter from who it was received (as the ID is packed in the dish request), and a queue system of "receipts" allows the waiter to immediately deliver the dish to the customer, updating the order struct

2.4 Testing

For testing we used the Unity library, which lets you create test functions that check memory with TEST_ASSERT. We created a test function for kitchen and resource loading that use a test data folder. There is also one to test an unsatisfied customer and the relative change in score.